

Chapter 16

Logic Programming Languages

--Prolog

1.1 Introduction

The logic paradigm

Answer a question via search for a solution

The logic paradigm is dramatically different from the other three main programming paradigms. The logic paradigm fits extremely well when applied in problem domains that deal with the extraction of knowledge from basic facts and relations. The logical paradigm seems less natural in the more general areas of computation.

Logic programming languages are languages that are based on symbolic logic and used to prove arguments. For instance, suppose you have the fact "Socrates is a man." Moreover, suppose you are given the rule: "All men are mortal." One can then infer that: "Socrates is mortal." The idea is to have a computer program infer this conclusion automatically.

Example problem: Murder Mystery

Four guests (Colonel Mustard, Professor Plum, Miss Scarlett, Ms. Green) attend a dinner party at the home of Mr. Boddy. Suddenly, the lights go out! When they come back, Mr. Boddy lies dead in the middle of the table. Everyone is a suspect.

Upon further examination, the following facts come to light:

- Mr Boddy was having an affair with Ms. Green.
- Professor Plum is married to Ms. Green.
- Mr. Boddy was very rich.
- Colonel Mustard is very greedy.
- Miss Scarlett was also having an affair with Mr. Boddy.

There are two possible motives for the murder: hatred and greed. Someone hates someone else if that other person is having an affair with his/her spouse. Someone is willing to commit murder if he/she is greedy and not rich, and the victim is rich.

Horn clauses

In almost all logic languages, axioms are written in a standard form known as Horn clause. A Horn clause consists of a head, or consequent term H , and a body consisting of terms B_i :

$$H \leftarrow B_1, B_2, \dots, B_n$$

The semantics of this statement are that when the B_i are all true, we can deduce that H is true as well. When reading aloud, we say “ H , if $B_1, B_2, \dots$, and B_n .” Horn clauses can be used to capture most, but not all, logical statements.

1.2 Some Simple Prolog Programs

A Prolog program is different from that of an imperative programming language. Whereas the latter consists of a sequence of instructions telling you how to compute something, the former basically provides a list of facts and rules. Once these facts and rules are typed in, one then queries the program with questions.

Here's an easy example of a Prolog program, one that mimics the previous set of facts and rules:

```
is_man(socrates).  
is_mortal(A) :- is_man(A).
```

Once the above program is typed in, we then run a Prolog interpreter and query the program. For instance, if we typed in the query:

```
?- is_mortal(socrates).
```

then the interpreter returns:

```
yes
```

indicating that socrates is indeed mortal.

Here's a more complicated example of a Prolog program:

```

edge(a, b).
edge(c, d).
edge(a, c).
edge(c, b).
path(X, X).
path(X, Y) :- edge(Z, Y), path(X, Z).

```

The above program represents a directed graph. The first four statements provide us some facts, namely, the directed edges of the graph. The identifiers a, b, c, d begin with lower case letters indicating that they represent actual values, in this case, particular vertices. The lines:

```

path(X, X).
path(X, Y) :- edge(Z, Y), path(X, Z).

```

basically provide a definition for a directed path: there is always a path from X back to itself; and if there is an edge from Z to Y and a path from X to Z, then there is a path from X to Y. Note that variables (as opposed to constants) are always represented by identifiers beginning with an uppercase letter.

Given this program, we can then query the program and get responses as follows:

```

?- edge(a, b).
true?
yes
?- edge(d, a).
No
?- path(a, b).
true?
yes

```

We can also ask questions like: "What vertices are neighbors of a?:"

```

?- edge(a, X).
X= b?
X= c
yes

```

Note that we must hit semicolon to get successive answers to our query.

1.3 Syntax of Prolog

As with other programming languages, Prolog programs consist of collections of statements, or clauses. All Prolog statements are constructed from **terms**. A Prolog term is a **constant**, a **variable**, or a **structure**.

Constants are either alphanumeric strings starting with lowercase letter or numbers. Examples: george, a, you_may_use_underscore, 234, 34.67.

Variables are alphanumeric strings (underscore allowed) starting with uppercase letter. Examples: X, Result, NumberOfReports, My_tax.

Structures consist of name (functor) and arguments (components) in parentheses. More formally, we define a structure to have the syntax:

functor(comma-separated list of parameters)

So, for example, the following are all examples of structures:

```
is_man(socrates).  
is_man(X).  
edge(a, b).  
edge(X, Y).
```

A statement is represented as a **clause** (some authors use the term Horn clause), comes in two flavors: **facts** and **rules**, each ending with a period. A fact looks like:

structure.

while a rule has syntax:

structure1 :- structure2.

or

structure1 :- list of comma-separated structures. Comma means and. The portion of a rule that is to the left of the :- is called the **head**, while the portion that is to the right is called the **body**. So, for instance, the following are all clauses:

```
edge(a, b).  
is_mortal(X) :- is_man(X).  
path(X, Y) :- edge(Z, Y), path(X, Z).
```

A rule such as:

```
s1 :- s2, s3.
```

corresponds to a logical statement "if s2 and s3 are true, then s1 is true." So the head of a clause corresponds to the consequent of a conditional (the "then"-part), while the body corresponds to the antecedent (the "if"-part). Moreover, the comma is to be interpreted as "and". Finally, we define a Prolog program as a sequence of clauses.

Facts and rules describe logical relationships among facts. These statements are the basis of the theorem-proving process. The theorem is in the form of a proposition that the system either proves or disproves. In Prolog, these propositions are called **goals**, or **queries**. For example,

```
is_man(fred).
```

Conjunctive propositions and propositions with variables are also legal goals. For example,

```
father(X, mike).
```

1.4 The Inferencing Process of Prolog

To prove a goal is true, the inferencing process must find

- o A fact in the database that matches the goal, e.g., `is_man(bob).`
- o A chain of inference rules and/or facts in the data repository that connect the goal to one or more facts in the database - a process known as resolution.

For example, if Q is the goal, then either Q must be found as a fact in the database or the inferencing process must find a fact $P1$ and a sequence of propositions $P2, P3, \dots, Pn$ such that

$P2:-P1$

$P3:-P2$

...

$Q:-Pn$

1.4.1 Resolution and Unification

Prolog uses two basic techniques in inferencing process: resolution and unification.

If a goal contains variables, Prolog uses unification. **Unification** is the pattern-matching process and is often used to associate a variable with a value. For example,

`father(bob).`

`man(X):- father(X).`

The goal: `man(X).`

`father(X)` and `father(bob)` will unify successfully, the following instantiation will take place.

`X= bob`

The unification rules for Prolog state that:

- A constant unifies only with itself.
- Two structures unify if and only if they have the same

functor and the same arity, and the corresponding arguments unify recursively.

- A variable unifies with anything. If the other thing has a value, then the variable is instantiated. If the other thing is an uninstantiated variable, then the two variables are associated in such a way that if either is given a value later, that value will be shared by both.

```
?- a = a.  
Yes           % constant unifies with itself  
?- a = b.  
No           % but not with another constant  
?- foo(a, b) = foo(a, b).  
Yes         % structures are recursively identical  
?- X = a.  
X = a       % variable unifies with constant  
Yes         % only once  
?- foo(a, b) = foo(X, b).  
X = a       % arguments must unify  
Yes         % only one possibility
```

The **Resolution** is an inference rule that allows inferred propositions to be computed from given propositions. Suppose there are two propositions with the forms:

$T \subset P2$

$Q1 \subset T$

Their meaning is that

$P2$ implies T , and T implies $Q1$, then $Q1 \subset P2$

For example, consider the two propositions:

older(joanne, jake) $\subset$ mother(joanne, jake)
wiser(joanne, jake) $\subset$ older(joanne, jake)

From these propositions, the following proposition can be constructed using resolution:

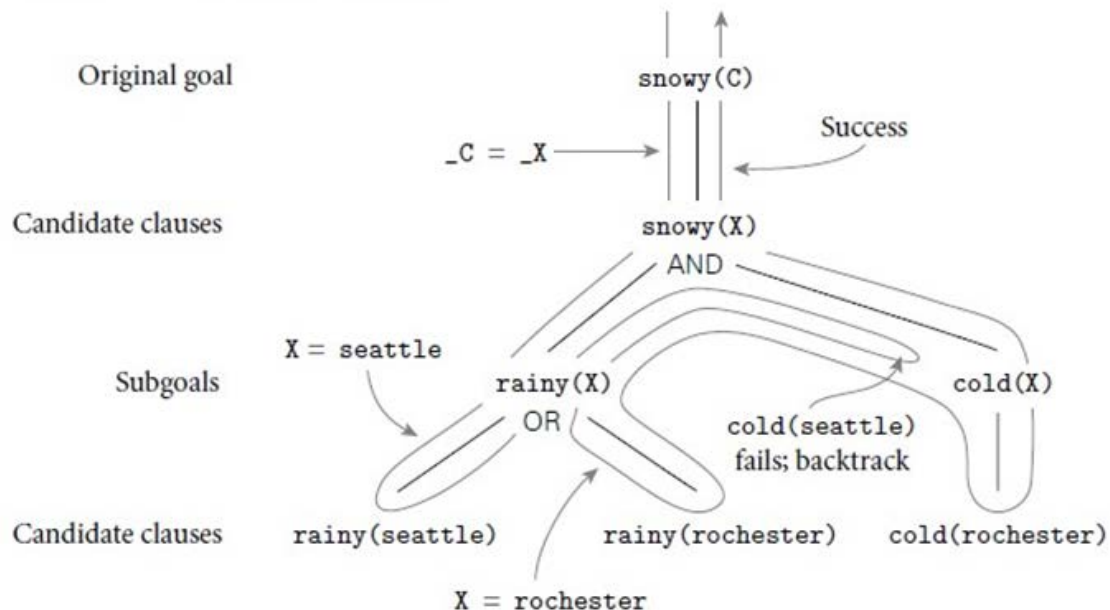
wiser(joanne, jake) $\subset$ mother(joanne, jake)

1.4.2 Backward Chaining

The Prolog interpreter requires a systematic way of proving or disproving a goal. The technique used by Prolog is called backward chaining. The basic idea is to start with the goal and work our ways backwards towards known facts:

```
prove(s)
for (all clauses c in the program)
    if (head of clause c matches s)
        apply resolution/unification to c to create new clause c'
        if (c' has no body)
            return is_fact(c')
        else
            return prove(s[1]) && prove(s[2]) &&... prove(s[n]),
                where the s[i] are the terms in the body of c'.
if (no heads of clauses match s)
    return no

rainy(seattle).
rainy(rochester).
cold(rochester).
snowy(X) :- rainy(X), cold(X).
```



If we assume that the && operator is short-circuited, then we see that as soon as we run into a non-provable structure, we simply "back up" in our process.

We remark that the above procedure is really nothing more than a depth-first search tree traversal.

Problem 1: Suppose we have clauses:

```
father(mike, greg) .
father(mike, marcia) .
mother(carol, greg) .
mother(carol, marcia) .
sibling(X, Y) :- father(F, X), father(F, Y), mother(M, X),
    mother(M, Y), X\==Y. %clause 5
```

Prove the term `sibling(greg, marcia)` using backward chaining.

1. `sibling(greg, marcia):- father(F, greg), father(F, marcia), mother(M, greg), mother(M, marcia), greg \== marcia.`
 2. `father(mike, greg).`
 3. `father(mike, marcia).`
 4. `mother(carol, greg).`
 5. `mother(carol, marcia).`
 6. `greg \== marcia.`
-

1.4.3. Tracing a Program

It is possible to "debug" a Prolog program by using the `trace` command. For example,

We can observe the execution of the program using `trace`:

```
{trace}
| ?- sibling(greg, marcia).
    1      1 Call: sibling(greg,marcia) ?
    2      2 Call: father(_79,greg) ?
    2      2 Exit: father(mike,greg) ?
    3      2 Call: father(mike,marcia) ?
    3      2 Exit: father(mike,marcia) ?
```

```

4      2  Call: mother(_129,greg) ?
4      2  Exit: mother(carol,greg) ?
5      2  Call: mother(carol,marcia) ?
5      2  Exit: mother(carol,marcia) ?
6      2  Call: greg\==marcia ?
6      2  Exit: greg\==marcia ?
1      1  Exit: sibling(greg,marcia) ?

```

```

true ?
yes

```

Variables marked with an underscore (e.g. `_79`) are unnamed, internal variables that needed to store instantiated values. A line marked `Call` means that the program is prepared to use Unification and Resolution to find a match to the expression to some head of a clause. A line marked `Exit` means that a successful match has been found to be true. Essentially, the `Call` is making a recursive function call, while `Exit` means returning from such a recursive call.

1.5 Simple Arithmetic

Prolog supports simple arithmetic. To handle arithmetic, Prolog provides a built-in predicate, `is`, that unifies its first argument with the arithmetic value of its second argument. For example, here is a Prolog program that computes and returns the sum of `B` and 4:

```
foo(A, B) :- A is B + 4.
```

We can compute the sum of 7 and 4 like so:

```

?- [foo].
foo(A, 7).

```

Here's what's happening.

- The program takes the goal `foo(A, 7)` and compares it to the

head of each rule; in this case, there's only one rule, namely,
`foo(A,B) :- A is B + 4.`

- The program then notices that B should be instantiated to 7. So the body of the rule now reads: `A is 7 + 4`
- The program then instantiates A to be 11.
- Finally, the program, having determined that `11 is 7 + 4` is true, has determined the truth of `foo(11, 7)`, completing the execution of the program.

It's possible to observe the above steps in slow motion using the trace command:

```
?- trace.
```

```
?- foo(A, 7).
```

```
| ?- foo(A,7).  
    1      1  Call: foo(_16,7) ?  
    2      2  Call: _16 is 7+4 ?  
    2      2  Exit: 11 is 7+4 ?  
    1      1  Exit: foo(11,7) ?  
  
A = 11  
  
yes  
{trace}  
| ?- 
```

Note that the `is` operator is not quite like an assignment statement. Due to the semantics of Prolog, the quantity to the right of the `is` must first be instantiated, and the quantity to the left of the `is` cannot be previously instantiated.

For instance, consider:

```
A is B / 17 + C.
```

If B and C are instantiated but A is not, then this statement will cause A to be instantiated with the indicated value. On the other hand, if either B or C is not instantiated or A is already instantiated, then the clause will not be satisfied and no instantiation of A can take place.

Problem 2: Explain why the following is not legal:

Sum is Sum + 1.

Example: Computing Distance Traveled

Here's an example (from the textbook) that computes the distance traveled by a car. The program first proclaims several facts regarding speed and time duration. The predicate distance accepts two parameters: the name of a car, and a variable to hold the resulting answer (distance traveled):

```
speed(ford, 100).
speed(chevy, 105).
speed(dodge, 95).
speed(volvo, 80).
time(ford, 20).
time(chevy, 21).
time(dodge, 24).
time(volvo, 24).
distance(X, Y) :- speed(X, Speed), time(X, Time), Y is Speed *
    Time.
```

The query is distance(chevy, Chevy_distance). Using trace, we can observe what happens:

```
{trace}
| ?- distance(chevy,Chevy_distance).
   1   1   Call: distance(chevy,_16) ?
   2   2   Call: speed(chevy,_85) ?
   2   2   Exit: speed(chevy,105) ?
   3   2   Call: time(chevy,_110) ?
   3   2   Exit: time(chevy,21) ?
   4   2   Call: _16 is 105*21 ?
   4   2   Exit: 2205 is 105*21 ?
   1   1   Exit: distance(chevy,2205) ?

Chevy_distance = 2205

yes
{trace}
| ?- 
```

Example: Factorial

Here's another example, this time a factorial. This predicate will accept two parameters. The first parameter is N, while the second parameter is the result, namely, N!:

```
factorial(0,1).
factorial(N,F) :- N1 is N-1,
                  factorial(N1,F1),
                  F is N * F1.
```

The first clause is the base case, namely, 0! is 1. The second clause is the recursive step. A trace of the program shows what's going on:

```
| ?- factorial(2, Result).
   1      1      Call: factorial(2,_16) ?
   2      2      Call: _88 is 2-1 ?
   2      2      Exit: 1 is 2-1 ?
   3      2      Call: factorial(1,_113) ?
   4      3      Call: _140 is 1-1 ?
   4      3      Exit: 0 is 1-1 ?
   5      3      Call: factorial(0,_165) ?
   5      3      Exit: factorial(0,1) ?
   6      3      Call: _193 is 1*1 ?
   6      3      Exit: 1 is 1*1 ?
   3      2      Exit: factorial(1,1) ?
   7      2      Call: _16 is 2*1 ?
   7      2      Exit: 2 is 2*1 ?
   1      1      Exit: factorial(2,2) ?

Result = 2 ?

(1 ms) yes
```

1.6 List Structures

Like Scheme, Prolog has the ability to work with lists. To specify a list, we use square brackets and commas: e.g.
[apple, prune, grape, kumquat]

We can specify that a list satisfies a certain criterion by using a predicate, e.g.:

```
new_list([apple, prune, grape, kumquat]).
```

Note that `new_list` is a name that was just made up. The above simply states the fact that the list `[apple, prune, grape, kumquat]` satisfies the predicate `new_list`.

We can split up a list using the pipe `|`. In doing so, we are basically specifying the head of a list and the remainder of the list. e.g.:

```
new_list([New_List_Head | New_List_Tail]).
```

This process is analogous to Schemes `CAR` and `CDR`.

```
[Head|Tail] = [mia, vincent, jules, yolanda].
```

Then

```
Head = mia  
Tail = [vincent,jules,yolanda]
```

Example: Append

To illustrate the use of a list structure, consider the following example. We want a predicate that simulates the Scheme version of the `APPEND` function.

The Prolog version will accept three parameters. The first two specify the lists that are to be smushed together, while the third records the resulting append:

```
append([], List, List).  
append([Head | List_1], List_2, [Head | List_3]) :-  
append(List_1, List_2, List_3).
```

Here is a sample trace:

```
?- trace.
```

```

?- append([bob, jo], [jake, darcie], Family).
1 1 Call: append([bob, jo], [jake, darcie], _G629) ?
2 2 Call: append([jo], [jake, darcie], _G692) ?
3 3 Call: append([], [jake, darcie], _G695) ?
3 3 Exit: append([], [jake, darcie], [jake, darcie]) ?
2 2 Exit: append([jo], [jake, darcie], [jo, jake, darcie]) ?
1 1 Exit: append([bob, jo], [jake, darcie], [bob, jo, jake, darcie]) ?

```

```
Family = [bob, jo, jake, darcie]
```

The first clause of the Prolog program is the base case, while the second clause is the recursive step. Note that these two clauses must be written in this order. (Why?)

The first clause (base case) basically says an empty list appended to a list results in that list. The second clause essentially tells us that, if appending *List_1* to *List_2* results in *List_3*, then prepending *Head* to *List_1* then appending the result to *List_2* is equivalent to prepending *Head* to *List_3*.

(The first clause (the base case) says that putting any list after an empty list results in that list. The second clause says that if putting **List₂** after **List₁** results in **List₃**, then putting **List₂** after a list that starts with **Head** followed by **List₁** results in a list that starts with **Head** followed by **List₃**.)

Compare this Prolog version to the Scheme version:

```

(define (append lis1 lis2)
  (cond
    ((null? lis1) (lis2))
    (#t (cons (car lis1) (append (cdr lis1) lis2)))))

```

Example: Member

This next example (from your textbook) implements a predicate *member*, which determines if a given symbol is a member of a given list. The predicate accepts two parameters: a given symbol,

followed by the list:

```
member(Element, [Element | _]).  
member(Element, [_ | List]) :- member(Element, List).
```

The underscore indicates an anonymous variable; it is used to mean that we don't care what instantiation it might get from unification.

Here's a trace to determine if a is a member of the list [b, c, d]:

```
?- trace.  
?- member(a, [b, c, d]).  
11 Call: member(a, [b, c, d]) ?  
22 Call: member(a, [c, d]) ?  
33 Call: member(a, [d]) ?  
44 Call: member(a, []) ?  
44 Fail: member(a, []) ?  
33 Fail: member(a, [d]) ?  
22 Fail: member(a, [c, d]) ?  
11 Fail: member(a, [b, c, d]) ?  
No.
```

Here's another trace to determine if a is a member of [b, c, a, d]:

```
?- member(a, [b, c, a, d]).  
11 Call: member(a, [b, c, a, d]) ?  
22 Call: member(a, [c, a, d]) ?  
33 Call: member(a, [a, d]) ?  
33 Exit: member(a, [a, d]) ?  
22 Exit: member(a, [c, a, d]) ?  
11 Exit: member(a, [b, c, a, d]) ?  
Yes
```


1.7 Arithmetic and Lists

It can be useful to know how long a list is. We'll give some examples of using lists together with arithmetic capabilities.

How long is a list? Here's a recursive definition.

- The empty list has length zero.
- A non-empty list has length $1 + \text{len}(T)$, where $\text{len}(T)$ is the length of its tail.

This definition is practically a Prolog program already. Here's the code we need:

```
len([],0).  
len([_|T],N) :- len(T,X), N is X+1.
```

This predicate works in the expected way. For example:

```
?- len([a,b,c,d,e,[a,b],g],X).  
X = 7
```

Now, this is quite a good program: it's easy to understand and efficient. But there is another method of finding the length of a list. We'll now look at this alternative, because it introduces the idea of accumulators. If you're used to other programming languages, you're probably used to the idea of using variables to hold intermediate results. An accumulator is the Prolog analog of this idea.

Here's how to use an accumulator to calculate the length of a list. We shall define a predicate `accLen` /3 which takes the following arguments.

accLen(List, Acc, Length)

Here `List` is the list whose length we want to find, and `Length` is its length (an integer). What about `Acc` ? This is the accumulator we will use to keep track of intermediate values for length (so it

will also be an integer). Here's what we do. When we call this predicate, we are going to give Acc an initial value of 0. We then recursively work our way down the list, adding 1 to Acc each time we find a head element, until we reach the empty list. When we reach the empty list, Acc will contain the length of the list. Here's the code:

```
accLen([_|T],A,L) :- Anew is A+1, accLen(T,Anew,L).
accLen([],A,A).
```

The base case of the definition unifies the second and third arguments. Why? Because this trivial unification is a nice way of making sure that the result, that is, the length of the list, is returned. When we reach the end of the list, the accumulator (the second variable) contains the length of the list. So we give this value (via unification) to the length variable (the third variable). Here's an example trace. You can clearly see how the length variable gets its value at the bottom of the recursion and passes it upwards as Prolog is coming out of the recursion.

```
accLen([a,b,c],0,L).
  Call: (6) accLen([a, b, c], 0, _G449)
  Call: (7) _G518 is 0+1
  Exit: (7) 1 is 0+1
  Call: (7) accLen([b, c], 1, _G449)
  Call: (8) _G521 is 1+1
  Exit: (8) 2 is 1+1
  Call: (8) accLen([c], 2, _G449)
  Call: (9) _G524 is 2+1
  Exit: (9) 3 is 2+1
  Call: (9) accLen([], 3, _G449)
  Exit: (9) accLen([], 3, 3)
  Exit: (8) accLen([c], 2, 3)
  Exit: (7) accLen([b, c], 1, 3)
  Exit: (6) accLen([a, b, c], 0, 3)
```

As a final step, we'll define a predicate which calls accLen for us, and gives it the initial value of 0:

```
leng(List,Length) :- accLen(List,0,Length).
```

So now we can pose queries like this:

```
?- leng([a,b,c,d,e,[a,b],g],X).
```

1.7. Deficiencies of Prolog

1.7.1. Problems with Resolution Order

Note that the order in which clauses and structures are considered is dictated by the order in which they are listed. This can, on occasion, create some problems. For example, recall our graph example from above:

```
edge(a, b) :- . % clause 1
edge(c, d) :- . % clause 2
edge(a, c) :- . % clause 3
edge(c, b) :- . % clause 4
path(X, X) :- . % clause 5
path(X, Y) :- edge(Z, Y), path(X, Z). % clause 6
```

Problem: Use backward resolution to prove `path(a, a)`.

Let's suppose, for the sake of argument, that we reversed the order of the last two clauses and also reverse the order of the terms in the body of clause 6:

```
path(X, Y) :- path(X, Z), edge(Z, Y). % reversed terms in body
path(X, X) :- . % and reserved order of clauses
```

From a logical standpoint, we haven't changed anything so we should expect to get the same answers. But let's say we again try to prove something like `path(a,a)`. Then, according to the backward chaining algorithm, we use resolution to generate the rule:

```
path(a, a) :- path(a, Z), edge(Z, a).
```

According to the backward chaining algorithm, we then try to

resolve the term `path(a, Z)`. The problem is that the first clause whose head matches that term uses the exact same rule:

```
path(a, Z) :- path(a, Z'), edge(Z', Z).
```

Clearly, then, we've run into an infinite regress. This points out a fundamental flaw with Prolog. Changing the order of terms or of clauses really shouldn't affect the semantics of the program. But it does. This violates a basic principle of logic programming, namely, that the focus should be on what a program does, not on how it does it.

There are ways of getting around these shortcomings. But each of these has shortcomings of their own:

- One technique is to try to detect such an infinite regress. In an example like above this was easy to do, but in general this is a very hard problem.
- Another approach is to use a breadth-first search instead of a depth-first search. Unfortunately, doing so requires a lot more memory (an impractical amount, really) since the breadth of the typical resolution tree is quite wide.
- Finally, instead of starting with the goal and working our way backwards toward facts, we could go the other way around: start with facts, then work our way towards our goal. This is called forward chaining. However, doing so would again require a lot of memory since we'd potentially have to generate every possible goal. Such an enumeration is quite large.

This illustrates the delicate balance between design and implementation issues. Namely, from a design perspective, we don't like backward chaining because we have to worry about the order of clauses or terms. Unfortunately, from an implementation perspective, any approach other than backward chaining would be infeasible.

As a final comment on resolution order, note that we see a similar problem with something like the database language SQL. Like Prolog, SQL focuses on issues of what we seek, not how we seek them.

For instance, consider the query:

```
SELECT *  
FROM A, B, C  
WHERE A.foo = B.foo  
AND B.goo = C.goo
```

Then swapping the order of the terms in the WHERE clause should have no effect on the query. In other words, we expect the same result from:

```
SELECT *  
FROM A, B, C  
WHERE B.goo = c.goo  
AND A.foo = B.foo
```

However, although the results generated by the two SQL queries are generally going to be the same, the execution time for each could differ dramatically, depending primarily on the results of the joins. From a design perspective, the order of terms really shouldn't affect the program's performance, but in reality it does.

1.7.2 The Negation Operator

As you might expect, there is an operator not that is used for negation. However, its behavior is a bit different than one might expect. For one thing, not simply means "not provable." It does not mean "provably false". So, for example, suppose we have

the program:

```
takes_class(john, csci571).  
takes_class(mary, csci571).  
takes_class(joe, csci519).
```

and consider the query and response:

```
?- not takes_class(joe, csci571).
```

Yes

The response doesn't mean that Joe is not in Csci 571. Rather, it simply means that we cannot say with certainty that Joe is in Csci571. So unless we can be certain that our facts cover all class enrollments (this is what's known as the "closed-world assumption"), trying to prove a negative statement such as, "Joe is not in Csci 571" is problematic in Prolog.

This is just like our judicial system. A "not guilty" verdict doesn't mean the person is provably innocent; it simply means that the prosecution was unable to provide sufficient evidence to convict the defendant.

1.8 Closing Remarks

The main power of Prolog lies in its ability to help the programmer focus on "what"-type issues instead of the "how"-type issues prevalent in imperative programming languages. These "what"-type questions are exactly the same type of questions one considers when generating, for instance, SQL queries for a database. In fact, the connection between SQL and Prolog is not accidental for several implementation and design considerations for both languages are eerily similar. And there are several projects underway to marry these two related programming paradigms.

Summary and Concluding Remarks (from a different textbook)

In this chapter we have focused on the logic model of computing. Where an imperative program computes principally through iteration and side effects, and a functional program computes principally through substitution of parameters into functions, a logic program computes through the resolution of logical statements, driven by the ability to unify variables and terms.

Much of our discussion was driven by an examination of the principal logic language, Prolog, which we used to illustrate clauses and terms, resolution and unification, search/execution order, list manipulation, and high-order predicates for inspection and modification of the logic database.

Like imperative and functional programming, logic programming is related to constructive proofs. But where an imperative or functional program in some sense is a proof (of the ability to generate outputs from inputs), a logic program is a set of axioms from which the computer attempts to construct a proof. And where imperative and functional programming provide the full power of Turing machines and lambda calculus, respectively (ignoring hardware-imposed limits on arithmetic precision, disk and memory space, etc.), Prolog provides less than the full generality of resolution theorem proving, in the interests of time and space efficiency. At the same time, Prolog extends its formal counterpart with true arithmetic, I/O, imperative control flow, and higher-order predicates for self-inspection and modification.

Like Lisp/Scheme, Prolog makes heavy use of lists, largely because they can easily be built incrementally, without the need to allocate and then modify state as separate operations. And like Lisp/Scheme (but unlike ML and its descendants), Prolog is

homoiconic: programs look like ordinary data structures, and can be created, modified, and executed on the fly.

As we stressed in Chapter 1, different models of computing are appealing in different ways. Imperative programs more closely mirror the underlying hardware, and can more easily be “tweaked” for high performance. Purely functional programs avoid the semantic complexity of side effects, and have proven particularly handy for the manipulation of symbolic (nonnumeric) data. Logic programs, with their highly declarative semantics and their emphasis on unification, are well suited to problems that emphasize relationships and search. At the same time, their de-emphasis of control flow can lead to inefficiency. At the current state of the art, computers have surpassed people in their ability to deal with low level details (e.g., of instruction scheduling), but people are still better at inventing good algorithms.

As we also stressed in Chapter 1, the borders between language classes are often very fuzzy. The backtracking search of Prolog strongly resembles the execution of generators in Icon. Unification in Prolog resembles (but is more powerful than) the pattern-matching capabilities of ML and Haskell. (Unification is also used for type checking in ML and Haskell, and for template instantiation in C++, but those are compile-time activities.)

There is much to be said for programming in a purely functional or logic-based style. While most Scheme and Prolog programs make some use of imperative language features, those features tend to be responsible for a disproportionate share of program bugs. At the same time, there seem to be programming tasks—graphical I/O, for example—that are almost impossible to accomplish without side effects.